

Java tools: generics and interfaces

Stacks of more than one type of thing, promises the compiler enforces

AGENDA

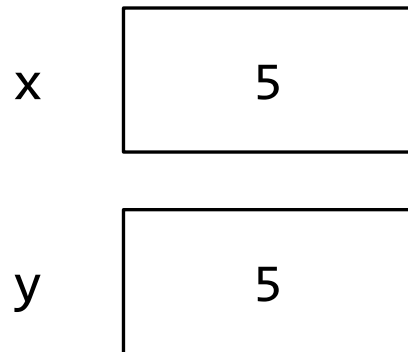
- primitives and references: what a variable actually holds
- generics: one Stack class that works for any type
- wrapper types and autoboxing
- interfaces: promises the compiler enforces

WHAT DOES A VARIABLE HOLD?

- A primitive variable holds the value itself. A reference variable holds an arrow to an object.

primitives: int, double, char, boolean, long, ...

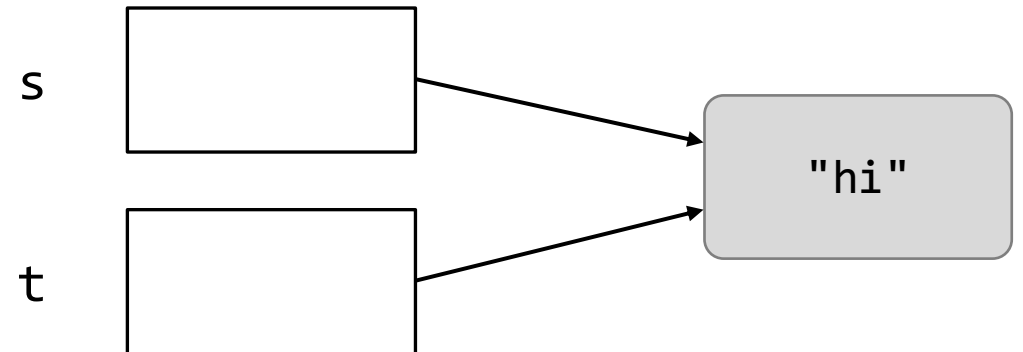
```
1 int x = 5;  
2 int y = x;
```



two independent copies of the value 5

everything else: String, Scanner, Stack, Node, arrays, ...

```
1 String s = "hi";  
2 String t = s;
```



two arrows to one object

WHY THAT DISTINCTION MATTERS TODAY

- Primitives are not objects. They have no methods: you can't write `5.equals(...)`.
- Reference variables can be null. A primitive never can: an `int` is always some number.
- `==` on primitives compares values. `==` on references compares arrows.
 - which is why strings need `.equals`
- Generics, coming up in a moment, only work with reference types. That turns out to matter.

THE PROBLEM

- Last time we built a stack. It holds strings, and only strings.

```
1 public class StackOfStrings {  
2     private class Node {  
3         String item;  
4         Node next;  
5     }  
6     public void push(String item) { ... }  
7     public String pop() { ... }  
8 }
```

- What if we want a stack of ints? Or a stack of Scanners? Or a stack of stacks?

THE OBVIOUS FIX: WRITE IT AGAIN

StringStack.java

```
1 public class StringStack {  
2     private String[] stack;  
3     private int n;  
4  
5     public void push(String item)  
6     public String pop()  
7     public boolean isEmpty()  
8 }
```

IntStack.java

```
1 public class IntStack {  
2     private int[] stack;  
3     private int n;  
4  
5     public void push(int item)  
6     public int pop()  
7     public boolean isEmpty()  
8 }
```

- Same algorithm, twice. The only difference is the type.
- Now do it again for Scanner. And again for Shape. And again for every type anyone ever writes.
- Copy-paste code is a maintenance nightmare: fix a bug in one, fix it in all of them.

WHAT WE WANT

- Write the class once, with a blank where the type goes. Let the client fill in the blank.

```
1 public class Stack<T> {  
2  
3     public void push(T item) { ... }  
4  
5     public T pop() { ... }  
6 }
```



T is a placeholder for a type. The client says what it really is.

- This is called a generic class. T is a type parameter.
- The textbook uses Item; T is the common convention, and it looks like a placeholder.

T GOES EVERYWHERE THE TYPE WENT

```
1  public class GenericStack<T> {  
2  
3      private class Node {  
4          T item;  
5          Node next;  
6      }  
7      private Node first = null;  
8  
9      public void push(T item) {  
10         Node oldFirst = first;  
11         first = new Node();  
12         first.item = item;  
13         first.next = oldFirst;  
14     }  
15  
16     public T pop() { ... }  
17 }
```

- Every place StackOfStrings said String, this says T.
- The push code is identical. Nothing about the algorithm cared what was in the box.
- One class. Every type.

THE CLIENT FILLS IN THE BLANK

```
1  GenericStack<String> words = new GenericStack<String>();
2  words.push("to");
3  words.push("be");
4  String w = words.pop();           // no cast needed
5
6  // <> means "same as the left side"
7  GenericStack<Shape> shapes = new GenericStack<>();
8  shapes.push(new Circle(1.0));
9
10 words.push(42);                   // compile error: 42 is not a String
```

- The compiler now knows what comes out of pop(), so no casting.
- It also knows what may go in. Pushing the wrong type fails to compile, instead of blowing up later.

BUT WHAT ABOUT int?

```
1 GenericStack<int> numbers = new GenericStack<>(); // does not compile
```

- A type parameter must be a reference type. int is a primitive, so it isn't allowed.
- Why: underneath, the compiler turns T into Object and inserts casts for you. Primitives are not Objects, so there is nothing to cast.
- (The technical name is type erasure. You don't need it today; you need the consequence.)
- So: is a stack of numbers just impossible?

WRAPPER TYPES

- Every primitive has a matching class whose objects hold one value.

primitive	wrapper class
int	Integer
double	Double
char	Character
boolean	Boolean
long	Long

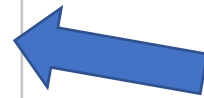
```
1  GenericStack<Integer> nums =  
2      new GenericStack<>();  
3  
4  nums.push(17);  
5  int sum = nums.pop() + nums.pop();
```

- Autoboxing: Java converts int to Integer on the way in, and back on the way out, automatically.
- push(17) really means push(Integer.valueOf(17)). You just don't have to write it.

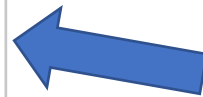
== ON STRINGS LIES TO YOU

- The danger isn't that == is always wrong. It's that == is often right, until it isn't.

```
1 String a = "hello";
2 String b = "hello";
3 a == b           // true
4
5 String part = "hel";
6 String d = part + "lo";
7 a == d           // false
8 a.equals(d)      // true
```



works! so you conclude == is fine

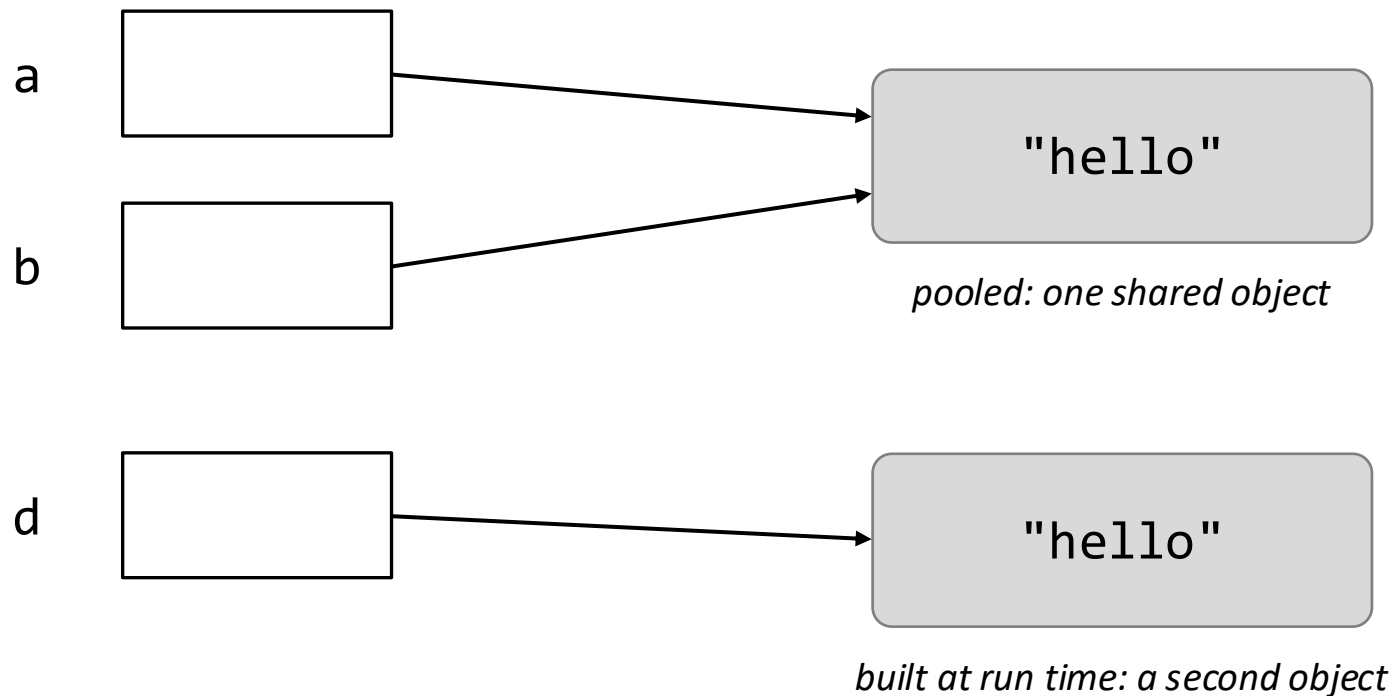


same letters. Different answer.

- Your test with typed-in literals passes. The version that reads real input fails. Same code.

WHY: THE STRING POOL

- Java reuses one object for identical text it can see while compiling. Anything built while the program runs is a new object.



- `==` asks: are these two arrows pointing at the same object?
 - `.equals` asks: do these two objects hold the same characters?
 - You almost always mean the second one.
- Anything a user types, anything read from a file, anything built with `+` at run time: new object every time.

TWO TRAPS THAT COME WITH IT

- Wrappers are objects. So == compares arrows, exactly like it does for strings.

```
1 Integer a = 127, b = 127;  
2 a == b           // true   (Java caches small values)  
3  
4 Integer c = 128, d = 128;  
5 c == d           // false  (two separate objects!)  
6 c.equals(d)      // true   (same value)
```

the rule from the String
slide, all over again:
compare with .equals



- Second trap: unboxing null throws a NullPointerException. An Integer can be null; an int cannot.

```
1 Integer missing = null;  
2 int x = missing;    // NullPointerException
```

ONE ROUGH EDGE: GENERIC ARRAYS

- The linked version needed no changes. The array version hits a wall.

```
1 private T[] stack;  
2  
3 stack = new T[capacity];           // does not compile  
4  
5 stack = (T[]) new Object[capacity]; // what we write instead
```

- The compiler warns about that cast ("unchecked"). Here it is safe: only our own class touches this array.
- Same cause as Stack<int>: the type parameter isn't there at run time, so Java can't build an array of it.
- Every generic collection in Java does some version of this internally.

GENERIC: RECAP

- Write the class once with a type parameter: `class Stack<T>`
- The client picks the real type: `new GenericStack<String>()`
- The compiler checks what goes in and knows what comes out, so no casts.
- Type parameters must be reference types, so primitives go in as wrapper objects.
- Autoboxing makes that mostly invisible, except for `==` and `null`.

Interfaces

promises the compiler enforces

AN INTERFACE IS A LIST OF PROMISES

```
1 public interface Shape {  
2     double area();  
3     String name();  
4 }
```

- Method signatures, and nothing else. No bodies, no code.
- An interface is not a class. You cannot say `new Shape()`.
- It says: any class that implements me will have these two methods.

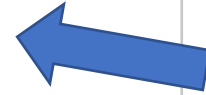
THE DIFFERENCE BETWEEN INTERFACES AND INHERITANCE

Implementing an interface gives you no code. None. Zero methods.

- This is the big difference from inheritance. Extend a class and you inherit its methods, ready to use.
- Implement an interface and you inherit an obligation: you must write those methods yourself.
- So why would anyone want that? Two slides from now.

MAKING THE PROMISE: implements

```
1 public class Circle implements Shape {  
2     private double radius;  
3  
4     public Circle(double radius) {  
5         this.radius = radius;  
6     }  
7  
8     public double area() {  
9         return Math.PI * radius * radius;  
10    }  
11  
12    public String name() {  
13        return "circle of radius " + radius;  
14    }  
15 }
```

 the promise

- Leave out either method and the class will not compile.
- That is the whole mechanism: the compiler holds you to it.
- Rectangle implements Shape too, with completely different code inside.

THE PAYOFF: AN INTERFACE IS A TYPE

```
1 Shape[] shapes = { new Circle(1.0),  
2                     new Rectangle(3.0, 4.0),  
3                     new Circle(2.5) };  
4  
5 double total = 0.0;  
6 for (Shape s : shapes) {  
7     System.out.println(s.name() + " " + s.area());  
8     total += s.area();  
9 }
```

guaranteed to exist. Shape
promised it, and the compiler
checked.



- Circle and Rectangle share no parent class. The interface is what lets them sit in one array.
- Write code against the interface and it works for classes that don't exist yet.

BOTH TOOLS AT ONCE

```
1  GenericStack<Shape> stack = new GenericStack<>();  
2  
3  for (Shape s : shapes) stack.push(s);  
4  
5  while (!stack.isEmpty()) {  
6      System.out.println(stack.pop().name());  
7  }
```

- A stack that holds anything shaped like a Shape.
- Generics say what kind of thing goes in the container. Interfaces say what that thing can do.

RULES AND NON-RULES

- No instance fields. Constants are allowed; per-object data is not.
- Methods are public automatically. Writing public is optional.
- Modern Java also allows default, static, and private methods in interfaces, with bodies. (Java 8 and 9 onward.)
- The big non-rule: a class may implement as many interfaces as it likes.
 - Multiple inheritance of classes causes the diamond problem: which parent's method do you get? Interfaces hand out no methods to conflict over.

INTERFACES YOU ALREADY DEPEND ON

- Comparable: a class that promises compareTo. It's how sorting works, and it's next week.
- Iterable: a class that promises an iterator, which is what lets a for-each loop walk it.
 - Our stack isn't Iterable yet, which is why you can't for-each over it. We'll fix that.
- List, Map, Set: Java's collections are interfaces, with several classes implementing each.

A WORD THAT MEANS TWO THINGS

- Earlier we said a stack's interface is its public menu: push, pop, isEmpty. That's the general sense, and it's what API means.
- Java also has a keyword interface, which is the specific mechanism from today.
- They're related but not the same. A class has an interface in the general sense whether or not it implements an interface in the Java sense.
- The Java interface keyword is *incredibly useful* for implementing the API concept.

ONE PROMISE, TWO WAYS TO KEEP IT

- Make the stack itself an interface. Then write it twice, the two ways we already know.

Stack.java

```
1 public interface Stack<T> {  
2     void push(T item);  
3     T pop();  
4     boolean isEmpty();  
5     int size();  
6 }
```

two classes that implement it

```
1 public class LinkedListStack<T>  
2     implements Stack<T> {  
3     // a chain of nodes  
4 }
```

```
1 public class ArrayStack<T>  
2     implements Stack<T> {  
3     // a resizing array  
4 }
```

- Totally different code inside. Identical promise outside.

HOT-SWAPPING THE IMPLEMENTATION

declared type: the promise



```
1 Stack<String> myStack = new LinkedListStack<>();  
2  
3 Stack<String> myStack = new ArrayStack<>();
```



actual object: one way of keeping it

- One word changes. Every other line of your program stays exactly the same.
- push, pop, isEmpty, size all still work: the interface promised them.
- The catch: you can only call what the interface promises. If ArrayStack had a bonus method, this variable couldn't see it.

WHY THAT'S WORTH THE TROUBLE

```
1  public static String reverse(Stack<String> stack, String line) {  
2      for (String word : line.split(" ")) stack.push(word);  
3      ...  
4  }
```

- This method doesn't care how the stack works. It just needs something that keeps the promise.
- Hand it a `LinkedListStack` or an `ArrayStack`. Same output, different machinery.
- So you can: try both, time both, and change your mind later without rewriting anything.
- This is the client / implementation / interface split from the stacks lecture, now enforced by the compiler.

WRAP-UP

- A variable holds either a value (primitive) or an arrow to an object (reference).
- Generics: one class, any reference type, checked at compile time.
- Wrapper types and autoboxing get primitives into generic containers.
- Interfaces: a promise of methods, enforced by the compiler, that also acts as a type.
- To come: sorting, where Comparable earns its keep.